Aaron Kromer describes another great use of this technique in his blog post, "Farewell JSON API Gems."[1] In it, he uses a helper method to return a matcher that describes the exact shape of a JSON response he expects from a particular API.

Defining Matcher Aliases

In *Passing One Matcher Into Another, on page 177*, you saw how RSpec defines a_value_within as an alias of the be_within matcher. It allows you to write expectations that read smoothly like the following one:

12-creating-custom-matchers/05/custom_matchers.rb
```
expect(results).to start_with a_value_within(0.1).of(Math::PI)
```

You can use the same techniques in your own projects. Just call alias_matcher with the name of the *new* matcher first, followed by the existing one (the same order you'd use with Ruby's alias_method):

12-creating-custom-matchers/05/custom_matchers.rb
```
RSpec::Matchers.alias_matcher :an_admin, :be_an_admin
```

This snippet defines a new method, an_admin, which wraps the existing be_an_admin matcher (a dynamic predicate matcher that calls admin?; see *Dynamic Predicates, on page 194*). The new matcher will use "an admin," rather than "be an admin," in its description and failure messages:

```
>> be_an_admin.description
=> "be an admin"
>> an_admin.description
=> "an admin"
```

The alias_matcher method can also take a block, for when you want something different from the matcher's Ruby method name for your descriptions. For instance, if you wanted an_admin to show up as a superuser in the output:

```
>> an_admin.description
=> "a superuser"
```

…you could define your alias like so:

12-creating-custom-matchers/05/custom_matchers.rb
```
RSpec::Matchers.alias_matcher :an_admin, :be_an_admin do |old_description|
  old_description.sub('be an admin', 'a superuser')
end
```

Dynamic predicate matchers like these are common targets for these kinds of aliases, since RSpec doesn't ship with its own aliases for them.

1. http://aaronkromer.com/blog/2014-09-29-farewell-json-api-gems.html